

【習題 3.3.3】

考慮[習題 2.3.1]有障礙物及陷阱的迷宮。試以同策略蒙地卡羅法來計算狀態值 $V\pi(s)$ 、狀態行動值 $q\pi(s,a)$ 及貪婪策略 π

答:

```
== 狀態值  $V\pi(s)$  ==
[' -64.35', ' -71.06', ' -81.22', ' -69.55', ' -62.13']
[' -67.27', ' -78.25', ' -75.14', ' -76.40', ' -63.53']
[' -69.19', ' -70.74', ' -76.49', ' -68.00', ' -61.25']
[' -80.55', ' -69.82', ' -63.18', ' 0', ' 0']
[' -95.21', ' -75.37', ' -52.49', ' -25.17', ' G']

== 狀態行動值  $q\pi(s,a)$  ==
State [0, 0]: {↑:-62.09, ↓:-64.81, ←:-62.1, →:-68.4}
State [0, 1]: {↑:-68.51, ↓:-75.45, ←:-62.13, →:-78.17}
State [0, 2]: {↑:-78.14, ↓:-111.02, ←:-68.49, →:-67.07}
State [0, 3]: {↑:-66.85, ↓:-73.52, ←:-78.03, →:-59.89}
State [0, 4]: {↑:-60.19, ↓:-61.54, ←:-67.05, →:-59.75}
State [1, 0]: {↑:-62.21, ↓:-66.79, ←:-64.89, →:-75.18}
State [1, 1]: {↑:-68.68, ↓:-68.33, ←:-65.08, →:-110.98}
State [1, 2]: {↑:-75.53, ↓:-74.93, ←:-77.41, →:-72.66}
State [1, 3]: {↑:-67.35, ↓:-65.88, ←:-111.5, →:-61.27}
State [1, 4]: {↑:-60.2, ↓:-58.98, ←:-73.53, →:-61.36}
State [2, 0]: {↑:-64.9, ↓:-77.52, ←:-66.49, →:-67.89}
State [2, 1]: {↑:-75.27, ↓:-67.31, ←:-66.76, →:-73.56}
State [2, 2]: {↑:-111.0, ↓:-60.99, ←:-68.52, →:-65.65}
State [2, 3]: {↑:-73.58, ↓:-65.4, ←:-73.82, →:-59.2}
State [2, 4]: {↑:-61.09, ↓:-59.26, ←:-65.44, →:-59.24}
State [3, 0]: {↑:-66.85, ↓:-110.94, ←:-77.26, →:-67.49}
State [3, 1]: {↑:-68.18, ↓:-72.62, ←:-77.69, →:-60.86}
State [3, 2]: {↑:-73.6, ↓:-50.85, ←:-67.24, →:-61.06}
State [3, 3]: 0
State [3, 4]: 0
State [4, 0]: {↑:-78.95, ↓:-116.07, ←:-113.05, →:-73.68}
State [4, 1]: {↑:-67.01, ↓:-72.58, ←:-111.34, →:-50.71}
State [4, 2]: {↑:-61.27, ↓:-51.11, ←:-72.5, →:-24.57}
State [4, 3]: {↑:-25.58, ↓:-24.8, ←:-50.75, →:0.0}
State [4, 4]: G

== 貪婪策略  $\pi$  ==
['↑', '←', '→', '→', '→']
['↑', '←', 'T', '→', '↓']
['↑', '←', '↓', '→', '→']
['↑', '→', '↓', 'O', 'O']
['T', '→', '→', '→', 'G']
```

【習題 3.3.4】

考慮[習題 2.3.1]有障礙物及陷阱的迷宮。試以同策略 ϵ -貪婪蒙地卡羅法來計算最佳狀態值 $V^*(s)$ 、最佳狀態行動值 $q^*(s,a)$ 及最佳策略 π^* ，其中 $\epsilon=0.1$

答:

== 最佳狀態值 $V^*(s)$ ==

```
['-10.1136', '-11.2607', '-10.7464', '-10.5279', '-9.2427']
['-8.0585', '-5.8090', '-59.6079', '-8.4253', '-8.1646']
['-7.2976', '-4.5678', '-3.4891', '-6.3126', '-7.2126']
['-4.5389', '-3.4768', '-2.3574', '0', '0']
['-59.6079', '-2.3784', '-1.1494', '0.0000', 'G']
```

== 狀態-動作值 $Q^*(s,a)$ ==

```
state = [0, 0] → Q = ['-16.76', '-15.70', '-17.32', '-10.11'] → greedy = ['↓']
state = [0, 1] → Q = ['-13.67', '-16.14', '-22.13', '-11.26'] → greedy = ['↓']
state = [0, 2] → Q = ['-11.87', '-13.56', '-10.75', '-66.83'] → greedy = ['→']
state = [0, 3] → Q = ['-12.88', '-14.08', '-10.53', '-13.34'] → greedy = ['→']
state = [0, 4] → Q = ['-14.74', '-15.55', '-15.95', '-9.24'] → greedy = ['↓']
state = [1, 0] → Q = ['-10.41', '-15.89', '-11.76', '-8.06'] → greedy = ['↓']
state = [1, 1] → Q = ['-10.80', '-12.74', '-60.12', '-5.81'] → greedy = ['↓']
state = [1, 2] → Q = ['-10.03', '-13.60', '-9.95', '-6.25'] → greedy = ['↓']
state = [1, 3] → Q = ['-60.81', '-10.70', '-12.49', '-8.43'] → greedy = ['↓']
state = [1, 4] → Q = ['-15.24', '-12.78', '-15.96', '-8.16'] → greedy = ['↓']
state = [2, 0] → Q = ['-12.41', '-9.63', '-7.69', '-7.30'] → greedy = ['↓']
state = [2, 1] → Q = ['-8.99', '-11.04', '-8.46', '-4.57'] → greedy = ['↓']
state = [2, 2] → Q = ['-6.71', '-58.87', '-7.29', '-3.49'] → greedy = ['↓']
state = [2, 3] → Q = ['-6.31', '-12.35', '-8.55', '-8.22'] → greedy = ['←']
state = [2, 4] → Q = ['-7.21', '-12.06', '-8.18', '-8.27'] → greedy = ['←']
state = [3, 0] → Q = ['-6.99', '-8.27', '-4.54', '-58.13'] → greedy = ['→']
state = [3, 1] → Q = ['-7.02', '-6.41', '-3.48', '-5.15'] → greedy = ['→']
state = [3, 2] → Q = ['-4.63', '-6.83', '-3.45', '-2.36'] → greedy = ['↓']
state = [3, 3] → Obstacle
state = [3, 4] → Obstacle
state = [4, 0] → Q = ['-58.69', '-7.31', '-7.70', '-59.12'] → greedy = ['↑']
state = [4, 1] → Q = ['-60.65', '-5.08', '-2.38', '-6.71'] → greedy = ['→']
state = [4, 2] → Q = ['-5.67', '-3.56', '-1.15', '-2.38'] → greedy = ['→']
state = [4, 3] → Q = ['-2.42', '-1.19', '0.00', '-1.15'] → greedy = ['→']
state = [4, 4] → Goal
```

== 最佳策略 $\pi^*(s)$ ==

```
['↓', '↓', '→', '→', '↓']
['↓', '↓', 'T', '↓', '↓']
['↓', '↓', '↓', '←', '←']
['→', '→', '↓', '0', '0']
['T', '→', '→', '→', 'G']
```

【習題 3.3.5】

考慮[習題 2.3.1]有障礙物及陷阱的迷宮。試以異策略 ϵ -貪婪蒙地卡羅法來計算最佳狀態值 $V^*(s)$ 、最佳狀態行動值 $q^*(s,a)$ 及最佳策略 π^* ，其中 $\epsilon=0.1$ 。假設行為策略 b 是選擇 4 個方向行動的機率都一樣。

答：

```
episode = 5000
episode = 10000
```

==最佳狀態值 $V^*(s)$ ==

```
['-7.1047' '-8.0227' '-6.8016' '-6.3287' '-6.6654']
['-6.2044' '-6.4030' '-56.0418' '-5.7471' '-6.2510']
['-5.2647' '-4.3559' '-3.3303' '-4.9953' '-5.5670']
['-4.1968' '-3.3099' '-2.2574' '0' '0']
['-56.4530' '-2.2459' '-1.1429' '0.0000' 'G']
```

== 狀態-動作值 $Q^*(s, a)$ ==

```
state = [0, 0] → Q = ['-7.41', '-7.31', '-8.05', '-7.10'] → greedy = ['↓']
state = [0, 1] → Q = ['-8.20', '-8.35', '-8.02', '-11.93'] → greedy = ['→']
state = [0, 2] → Q = ['-7.41', '-11.35', '-6.80', '-54.10'] → greedy = ['→']
state = [0, 3] → Q = ['-11.29', '-7.00', '-7.33', '-6.33'] → greedy = ['↓']
state = [0, 4] → Q = ['-7.42', '-8.03', '-8.75', '-6.67'] → greedy = ['↓']
state = [1, 0] → Q = ['-6.99', '-9.80', '-10.42', '-6.20'] → greedy = ['↓']
state = [1, 1] → Q = ['-7.75', '-8.46', '-54.08', '-6.40'] → greedy = ['↓']
state = [1, 2] → Q = ['-57.18', '-57.07', '-56.04', '-56.99'] → greedy = ['→']
state = [1, 3] → Q = ['-55.79', '-10.39', '-6.98', '-5.75'] → greedy = ['↓']
state = [1, 4] → Q = ['-7.03', '-6.87', '-7.57', '-6.25'] → greedy = ['↓']
state = [2, 0] → Q = ['-5.97', '-7.04', '-5.26', '-5.55'] → greedy = ['→']
state = [2, 1] → Q = ['-6.01', '-16.99', '-7.18', '-4.36'] → greedy = ['↓']
state = [2, 2] → Q = ['-5.09', '-55.51', '-6.01', '-3.33'] → greedy = ['↓']
state = [2, 3] → Q = ['-5.00', '-9.50', '-7.01', '-5.52'] → greedy = ['←']
state = [2, 4] → Q = ['-5.57', '-7.33', '-6.54', '-6.31'] → greedy = ['←']
state = [3, 0] → Q = ['-5.16', '-5.95', '-4.20', '-54.59'] → greedy = ['→']
state = [3, 1] → Q = ['-5.10', '-5.23', '-3.31', '-3.91'] → greedy = ['→']
state = [3, 2] → Q = ['-4.26', '-8.05', '-3.94', '-2.26'] → greedy = ['↓']
state = [3, 3] → Obstacle
state = [3, 4] → Obstacle
state = [4, 0] → Q = ['-56.45', '-57.28', '-57.18', '-56.96'] → greedy = ['←']
state = [4, 1] → Q = ['-54.15', '-4.19', '-2.25', '-3.31'] → greedy = ['→']
state = [4, 2] → Q = ['-3.32', '-3.34', '-1.14', '-2.26'] → greedy = ['→']
state = [4, 3] → Q = ['-2.26', '-1.15', '0.00', '-1.14'] → greedy = ['→']
state = [4, 4] → Goal
```

==最佳策略 $\pi^*(s)$ ==

```
['↓', '→', '↓', '↓', '↓', '↓']
['↓', '↓', '↓', '↓', '↓', '↓']
['→', '↓', '↓', '↓', '←', '←']
['→', '→', '↓', '↓', '0', '0']
['T', '→', '→', '→', '→', 'G']
```

【習題 3.3.3】

試以同策略蒙地卡羅法來計算狀態值 $V\pi(s)$ 、狀態行動值 $q\pi(s,a)$ 及貪婪策略 π^* ，程式碼如下：

```
import numpy as np

nrow, ncol = 5, 5
discount = 0.95
```

```

eps = 1.0e-5
action_word = np.array(["↑", "↓", "←", "→"])
action_num = 4
action_prob = np.ones(action_num) / action_num # 初始為均勻隨機策略

obstacle = [[3, 3], [3, 4]]
trap = [[1, 2], [4, 0]]
terminal = [4, 4]

state_value = np.zeros((nrow, ncol)) # V(s)
state_action_value = np.zeros((nrow, ncol,
action_num)) # Q(s,a)
state_visit_num = np.zeros((nrow, ncol),
dtype=int) # V(s) 訪問次數
state_return_accum = np.zeros((nrow, ncol)) #
V(s) 回報累積
state_action_visit_num = np.zeros((nrow, ncol,
action_num), dtype=int) # Q(s,a) 訪問次數
state_action_return_accum = np.zeros((nrow, ncol,
action_num)) # Q(s,a) 回報累積
greedy_action = np.zeros((nrow, ncol, action_num)) # 最終貪婪策略

# 判斷是否到達終點
def is_terminal(state):
    return state == terminal
# 判斷是否為障礙格子
def is_obstacle(state):
    return state in obstacle
# 判斷是否為陷阱格子
def is_trap(state):
    return state in trap

# 隨機選取初始狀態，避開終點、障礙與陷阱
def get_initial():
    while True:
        r = np.random.randint(0, nrow)

```

```

        c = np.random.randint(0, ncol)
        if not is_terminal([r, c]) and not is_obstacle([r,
c]):
            return [r, c]

# 根據動作計算下一狀態與獎勵
def step(state, action):

    r, c = state
    next_r, next_c = r, c

    if is_terminal([r, c]): # 終點不動
        return {"next_state": [r, c], "reward": 0.0,
"trap": False}

    # 根據動作更新位置
    if action == 0: # ↑
        next_r = max(r - 1, 0)
    elif action == 1: # ↓
        next_r = min(r + 1, nrow - 1)
    elif action == 2: # ←
        next_c = max(c - 1, 0)
    elif action == 3: # →
        next_c = min(c + 1, ncol - 1)

    next_state = [next_r, next_c]

    # 特殊狀態判斷
    if is_obstacle(next_state):
        return {"next_state": [r, c], "reward": -1.0,
"trap": False} # 撞到障礙物停下
    elif is_trap(next_state):
        return {"next_state": [0, 0], "reward": -50.0,
"trap": True} # 掉入陷阱回到起點
    elif is_terminal(next_state):
        return {"next_state": next_state, "reward": 0.0,
"trap": False} # 終點
    else:

```

```

        return {"next_state": next_state, "reward": -1.0,
                "trap": False} # 一般移動

# 蒙地卡羅隨機策略更新
episode_num = 10000
np.random.seed(42)
max_steps_per_episode = 1000

for episode in range(episode_num):
    state = get_initial()
    trajectory = [] # 用於存儲 (state, action, reward)
    steps = 0

    while not is_terminal(state) and steps <
max_steps_per_episode:
        steps += 1
        action = np.random.choice(action_num,
p=action_prob) # 根據隨機策略選動作
        step_info = step(state, action)
        next_state = step_info["next_state"]
        reward = step_info["reward"]
        trap_flag = step_info["trap"]

        trajectory.append((state[0], state[1], action,
reward))

        if trap_flag:
            state = [0, 0] # 掉陷阱回起點
        else:
            state = next_state

# 計算回報 Gt (從最後一步往回折扣累計)
Gt = 0.0
for s_r, s_c, a, r in reversed(trajectory):
    Gt = discount * Gt + r

# 更新狀態值 V(s)
state_visit_num[s_r, s_c] += 1

```

```

        state_return_accum[s_r, s_c] += Gt
        state_value[s_r, s_c] = state_return_accum[s_r,
s_c] / state_visit_num[s_r, s_c]

        # 更新狀態-行動值  $Q(s,a)$ 
        state_action_visit_num[s_r, s_c, a] += 1
        state_action_return_accum[s_r, s_c, a] += Gt
        state_action_value[s_r, s_c, a] =
state_action_return_accum[s_r, s_c, a] /
state_action_visit_num[s_r, s_c, a]

#處理特殊格子與貪婪策略
for r, c in obstacle:
    state_value[r, c] = 0
    state_action_value[r, c, :] = 0

# 根據  $Q(s,a)$  生成貪婪策略
for r in range(nrow):
    for c in range(ncol):
        if is_terminal([r, c]) or is_obstacle([r, c]):
            greedy_action[r, c, :] = 0
        else:
            q_vals = state_action_value[r, c, :]
            q_max = np.max(q_vals)
            idxs = np.where(np.abs(q_vals - q_max) <
eps)[0] # 可能有多個最優動作
            greedy_action[r, c, :] = 0
            greedy_action[r, c, idxs[0]] = 1 # 選其中一個最
優動作

which = lambda status: np.arange(len(status))[status]

print("\n=== 狀態值  $V_{\pi}(s)$  ===")
for i in range(nrow):
    row_display = []
    for j in range(ncol):
        if [i, j] in obstacle:
            row_display.append(" O ")

```

```

        elif [i, j] in trap:
            row_display.append(f"{state_value[i,j]:8.2f}")
        elif [i, j] == terminal:
            row_display.append(" G ")
        else:
            row_display.append(f"{state_value[i,j]:8.2f}")
    print(row_display)

print("\n=== 狀態行動值  $Q_{\pi}(s,a)$  ===")
for i in range(nrow):
    for j in range(ncol):
        s = [i, j]
        if s in obstacle:
            print(f"State {s}: O")
            continue
        elif s == terminal:
            print(f"State {s}: G")
            continue
        q_vals = np.round(state_action_value[i,j,:], 2)
        print(f"State {s}: {{↑:{q_vals[0]}}, ↓:{q_vals[1]}},
←:{q_vals[2]}, →:{q_vals[3]}}}")

print("\n=== 貪婪策略  $\pi$  ===")
for i in range(nrow):
    row_display = []
    for j in range(ncol):
        s = [i, j]
        if s in obstacle:
            row_display.append("O")
        elif s in trap:
            row_display.append("T")
        elif s == terminal:
            row_display.append("G")
        else:
            action_idx = np.argmax(greedy_action[i,j,:])
            row_display.append(str(action_word[action_idx]))
    ))
    print(row_display)

```


【習題 3.3.4】

試以同策略 ϵ -貪婪蒙地卡羅法來計算最佳狀態值 $V^*(s)$ 、最佳狀態行動值 $q^*(s,a)$ 及最佳策略 π^* ，程式碼如下：

```
import numpy as np

nrow, ncol = 5, 5
discount = 0.95
prob_eps = 0.1
max_steps = 100          #每回合最大步數
np.random.seed(543)

action = np.array([0, 1, 2, 3]) #0=左, 1=上, 2=右, 3=下
action_num = 4

S = [0, 0]
G = [4, 4]
O = [[3,3],[3,4]]
T = [[1,2],[4,0]]

# 值函數與統計資料初始化
state_value = np.zeros((nrow,ncol))          # V(s)
state_action_value = np.zeros((nrow,ncol,action_num)) # Q(s,a)
greedy_action = np.zeros((nrow,ncol,action_num)) # 優勢動作標記
visit_num = np.zeros((nrow,ncol,action_num), dtype=int) # 記錄每個 (s,a) 被訪問次數
return_accum = np.zeros((nrow,ncol,action_num)) # 累計回報，用於計算 Monte Carlo 平均

# 判斷是否到達終止狀態
def is_terminal(state):
    return state == G
```

```

# 隨機初始化起始狀態（不在障礙或目標）
def get_initial():
    while True:
        i,j = np.random.randint(0,nrow),
np.random.randint(0,ncol)
        state = [i,j]
        if state == G or state in O:
            continue
        return state

# 根據動作進行一步
def step(state, a):
    x,y = state
    # 計算下一個位置
    if a==0: nx,ny = x,y-1
    elif a==1: nx,ny = x-1,y
    elif a==2: nx,ny = x,y+1
    elif a==3: nx,ny = x+1,y
    else: raise ValueError("Invalid action")

    # 邊界限制
    nx = np.clip(nx,0,nrow-1)
    ny = np.clip(ny,0,ncol-1)
    next_state = [int(nx),int(ny)]

    # 若撞到障礙：停留原地，reward -1
    if next_state in O:
        return {"next_state": state[:], "reward": -1}

    # 若掉入陷阱：回到起點，reward -50
    elif next_state in T:
        return {"next_state": S[:], "reward": -50}

    # 到達目標
    elif next_state == G:
        return {"next_state": next_state, "reward": 0}

    # 一般移動，reward -1

```

```

else:
    return {"next_state": next_state, "reward": -1}

# 初始策略均勻分布（對非障礙與非目標狀態）
action_prob = np.zeros((nrow,ncol,action_num))
for i in range(nrow):
    for j in range(ncol):
        if [i,j] in O or [i,j]==G:
            continue
        action_prob[i,j,:] = 1/action_num # 均勻選擇

# 蒙地卡羅控制核心
episode_num = 50000 # 訓練回合數

for episode in range(episode_num):
    state_action_trajectory = [] # 記錄 (s,a) 序列
    reward_seq = [] # 記錄回報序列
    state = get_initial() # 隨機起始狀態
    steps = 0

    # 生成一個完整回合
    while not is_terminal(state) and steps < max_steps:
        prob = action_prob[state[0],state[1],:]
        if prob.sum() == 0: # 防止全 0
            prob = np.ones(action_num)/action_num
        move = int(np.random.choice(action,p=prob)) # 根據
策略選動作
        state_action_trajectory.append([state[0],state[1],
move])
        res = step(state,move)
        next_state = res["next_state"]
        reward = res["reward"]
        reward_seq.append(reward)
        state = next_state
        steps += 1

    # 每訪問蒙地卡羅回報更新
    Gt = 0

```

```

    for i in range(len(state_action_trajectory)-1,-1,-1):
        Gt = discount*Gt + reward_seq[i]    # 從最後一步回推折
扣回報
        k0,k1,k2 = state_action_trajectory[i]
        visit_num[k0,k1,k2] += 1
        return_accum[k0,k1,k2] += Gt
        state_action_value[k0,k1,k2] =
return_accum[k0,k1,k2]/visit_num[k0,k1,k2]    # Q(s,a) 更新

        # epsilon-greedy 策略更新
        q_values = state_action_value[k0,k1,:]
        index_max = np.where(q_values ==
np.max(q_values))[0]
        n1 = len(index_max)
        n2 = action_num - n1

        if n1 == action_num:    # 全部動作一樣好
            action_prob[k0,k1,:] = 1/action_num
        else:
            for k in range(action_num):
                if k in index_max:    # 最佳動作
                    action_prob[k0,k1,k] = (1-prob_eps)/n1
                else:
                    # 探索動作
                    action_prob[k0,k1,k] = prob_eps/n2

        state_value[k0,k1] = np.max(q_values)    # 更新 V(s)
        greedy_action[k0,k1,:] = 0
        greedy_action[k0,k1,index_max] = 1    # 標記最佳動
作

arrow_dict = {0:"←", 1:"↑", 2:"→", 3:"↓"}

print("\n=== 最佳狀態值 V*(s)===")
for i in range(nrow):
    row = []
    for j in range(ncol):
        state = [i,j]
        if state in 0:

```

```

        row.append("O")
    elif state == G:
        row.append("G")
    elif state in T:
        trap_val = -50 + discount * state_value[S[0],
S[1]] # 陷阱價值
        row.append(f"{trap_val:.4f}")
    else:
        row.append(f"{state_value[i,j]:.4f}")
    print(row)
print("\n=== 狀態-動作值  $Q^*(s,a)$  ===")
for i in range(nrow):
    for j in range(ncol):
        state = [i,j]
        if state == G:
            print(f"state = {state} → Goal")
        elif state in O:
            print(f"state = {state} → Obstacle")
        else:
            q_vals = [f"{state_action_value[i,j,k]:.2f}"
for k in range(action_num)]
            greedy_idx =
np.where(greedy_action[i,j,:]==1)[0]
            greedy_arrows = [arrow_dict[k] for k in
greedy_idx]
            print(f"state = {state} → Q = {q_vals} →
greedy = {greedy_arrows}")
print("\n=== 最佳策略  $\pi^*(s)$  ===")
for i in range(nrow):
    row = []
    for j in range(ncol):
        state = [i,j]
        if state == G:
            row.append("G")
        elif state in O:
            row.append("O")
        elif state in T:
            row.append("T")

```

```

        else:
            idx = np.where(greedy_action[i,j,:]==1)[0]
            arrows = "".join([arrow_dict[k] for k in idx])
            row.append(arrows)
    print(row)

```

【習題 3.3.5】

試以異策略 ϵ -貪婪蒙地卡羅法來計算最佳狀態值 $V^*(s)$ 、最佳狀態行動值 $q^*(s,a)$ 及最佳策略 π^* ，程式碼如下：

```

import numpy as np

which = lambda status: np.arange(len(status))[status]

nrow, ncol = 5, 5          # 地圖大小
discount = 0.95            # 折扣因子  $\gamma$ 
probab_eps = 0.1           # epsilon-greedy 策略的探索比例
eps = 1.0e-5               # 用於浮點數比較的小數
np.random.seed(543)        # 設定隨機種子，方便結果可重現

# Actions 動作編號對應方向
action_word = np.array(["left", "up", "right", "down"])
action = np.array([0, 1, 2, 3])
action_num = 4

START = [0, 0]              # 起點
GOAL = [4, 4]               # 目標
OBSTACLES = [[3, 3], [3, 4]] # 障礙物
TRAPS = [[1, 2], [4, 0]]    # 陷阱

# 獎勵設定
R_GOAL = 0.0
R_TRAP = -50.0
R_STEP = -1.0

```

```

# 動作對應的 row, col 變化
MOVE = {0: (0, -1), 1: (-1, 0), 2: (0, 1), 3: (1, 0)}

# ----- 初始化表格 -----
state_value = np.zeros((nrow, ncol))          # 狀態值
V(s)
state_action_value = np.zeros((nrow, ncol,
action_num))  # 狀態-動作值 Q(s,a)
W_accum = np.zeros((nrow, ncol, action_num))  #
importance sampling 權重累加器
greedy_action = np.zeros((nrow, ncol, action_num))  # 儲存
每個狀態的貪婪動作

# 目標策略與行為策略初始機率 (均勻分配)
target_action_prob = np.zeros((nrow, ncol, action_num))
behavior_action_prob = np.zeros((nrow, ncol, action_num))

for i in range(nrow):
    for j in range(ncol):
        if [i, j] in OBSTACLES or [i, j] == GOAL:
            continue
        target_action_prob[i, j, :] = 0.25
        behavior_action_prob[i, j, :] = 0.25

def is_goal(s):
    return s == GOAL

def in_obs(s):
    return s in OBSTACLES

def in_trap(s):
    return s in TRAPS

# 隨機選取非特殊格子作為初始狀態
def get_initial():
    while True:
        i = np.random.randint(0, nrow)

```

```

        j = np.random.randint(0, ncol)
        state = [i, j]
        if state not in OBSTACLES and state not in TRAPS
and state != GOAL:
            return state

# 執行一步動作，並處理陷阱、障礙、邊界、終點
def step(state, a_idx):
    """陷阱回到起點"""
    if in_trap(state):
        return START.copy(), R_TRAP, False

    dr, dc = MOVE[a_idx]
    nxt = [state[0] + dr, state[1] + dc]
    nxt[0] = int(np.clip(nxt[0], 0, nrow - 1)) # 限制 row
在範圍內
    nxt[1] = int(np.clip(nxt[1], 0, ncol - 1)) # 限制 col
在範圍內

    if in_obs(nxt):
        return state, R_STEP, False # 碰到障礙不動

    if is_goal(nxt):
        return nxt, R_GOAL, True # 到達目標

    if in_trap(nxt):
        return nxt, R_STEP, False # 陷阱不立即終止，交給
backward update 處理

    return nxt, R_STEP, False # 一般移動

# 蒙地卡羅控制
episode_num = 10000

for episode in range(episode_num):
    if (episode + 1) % 5000 == 0:
        print("episode =", episode + 1)

```



```

# 儲存單一回合的軌跡
state_trajectory = []
state_action_trajectory = []
reward_seq = []

state = get_initial()
state_trajectory.append(state.copy())
done = is_goal(state)

while not done:
    # 根據行為策略選擇動作
    prob = behavior_action_prob[state[0], state[1], :]
    move = int(np.random.choice(action, p=prob))
    next_state, reward, done = step(state, move)

    state_action_trajectory.append([state[0],
state[1], move])
    reward_seq.append(reward)
    state_trajectory.append(next_state.copy())
    state = next_state

# backward update
Gt = 0.0 # 回合回報初始化
W = 1.0 # importance sampling 初始權重
for t in
reversed(range(len(state_action_trajectory))):
    i, j, a = state_action_trajectory[t]
    Gt = discount * Gt + reward_seq[t] # 計算折扣回報

    # 更新 state-action 值，使用加權平均
    W_accum[i, j, a] += W
    Q_old = state_action_value[i, j, a]
    state_action_value[i, j, a] = Q_old + (W /
W_accum[i, j, a]) * (Gt - Q_old)

    # 更新目標策略 (epsilon-greedy)
    q_values = state_action_value[i, j, :]
    q_max = np.max(q_values)

```

```

index_max = which(np.abs(q_values - q_max) < eps)
n1 = len(index_max)
n2 = action_num - n1

if n1 == action_num:
    target_action_prob[i, j, :] = 1 / action_num
else:
    for k in range(action_num):
        if k in index_max:
            target_action_prob[i, j, k] = (1 -
prob_eps) / n1
        else:
            target_action_prob[i, j, k] = prob_eps
/ n2

# 更新狀態值為最大 Q 值
state_value[i, j] = q_max
# 更新貪婪動作
greedy_action[i, j, :] = 0
greedy_action[i, j, index_max] = 1

# 更新 importance sampling 權重
W *= target_action_prob[i, j, a] /
behavior_action_prob[i, j, a]
if W < eps:
    break

print("\n===最佳狀態值 V*(s)===")
V_display = np.empty((nrow, ncol), dtype=object)
for i in range(nrow):
    for j in range(ncol):
        if [i, j] in OBSTACLES:
            V_display[i, j] = " O "
        elif [i, j] == GOAL:
            V_display[i, j] = " G "
        else:
            # 陷阱也顯示對應 V(s)
            V_display[i, j] = f"{state_value[i, j]:.4f}"

```

```

for row in V_display:
    print(row)

print("\n=== 狀態-動作值  $Q^*(s,a)$  ===")
arrow_dict = {0: "←", 1: "↑", 2: "→", 3: "↓"}
for i in range(nrow):
    for j in range(ncol):
        state = [i, j]
        if state == GOAL:
            print(f"state = {state} → Goal")
        elif state in OBSTACLES:
            print(f"state = {state} → Obstacle")
        else:
            q_vals = [f"{state_action_value[i, j, k]:.2f}"
for k in range(action_num)]
            greedy_idx = np.where(greedy_action[i, j, :]
== 1)[0]
            greedy_arrows = [arrow_dict[k] for k in
greedy_idx] if len(greedy_idx) > 0 else ["?"]
            print(f"state = {state} → Q = {q_vals} →
greedy = {greedy_arrows}")

print("\n===最佳策略  $\pi^*(s)$  ===")
arrow = np.array(["←", "↑", "→", "↓"])
policy_display = np.empty((nrow, ncol), dtype=object)
for i in range(nrow):
    for j in range(ncol):
        s = [i, j]
        if s == GOAL:
            policy_display[i, j] = " G "
        elif s in OBSTACLES:
            policy_display[i, j] = " O "
        elif s in TRAPS:
            policy_display[i, j] = " T "
        else:
            idx_max = which(greedy_action[i, j, :] == 1)
            if len(idx_max) == 1:

```

```
        policy_display[i, j] = f"
{arrow[idx_max[0]]} "
        elif len(idx_max) > 1:
            policy_display[i, j] =
"".join(arrow[idx_max])
        else:
            policy_display[i, j] = " ? "
for row in policy_display:
    print(row)
```